DÉPÔT PLC · PLC STUDIO

Référentiel d’infrastructure

Un seul fichier, `infrastructure.jsonc`, décrit désormais tout le matériel et les installations que PLC Studio sait piloter. Quand l’infrastructure change, c’est le seul fichier à mettre à jour.

1	Synthèse	demande, résultat, chiffres clés
2	Démarche	cartographie, conception, relecture
3	Le fichier `infrastructure.jsonc`	contenu et notations
4	Comment PLC Studio s’en sert	générateur, validation, interface
5	Sites, appareils et logiciels	le cas du banc AUBO i10
6	Les outils `plc infra`	vérifier, lister, mesurer l’impact
7	Procédures de changement	quoi modifier, dans quel cas
8	Vérifications effectuées	
9	Points à décider	firmware et fichiers matériels
10	Fichiers, branche et commits	

Rapport établi le 15 octobre 2026. Branche `claude/referentiel-infrastructure` du dépôt `alexisaeq/plc`, partie de `claude/cartes-et-axes` (commit `5816df0`). Le test du robot AUBO est mis de côté, comme demandé. Les chemins cités sont relatifs à la racine du dépôt.

1 Synthèse

Demande. « Générer un fichier qui sert de référence au PLC Studio. Ce fichier donnera les outils au générateur pour s'adapter aux possibles changements d'infrastructures. Ainsi on aura seulement ce fichier à mettre à jour s'il y a des changements. » Rapport et travail dans une nouvelle branche.

Avant ce travail, ce que PLC Studio savait du matériel était dispersé : un catalogue de 1 100 lignes de JavaScript (`shared/catalog.js`), des limites recopiées dans la validation, le générateur, l'interface et la documentation, des bornes de variateurs codées dans le générateur C++, et aucune trace des installations réelles (IP du robot, logiciel du contrôleur, Ethernet de l'automate). Tout cela vit maintenant dans `infrastructure.jsonc`, à la racine du dépôt.

1 fichier de référence (947 lignes commentées en français)	**22** équipements décrits : 8 automates, 3 robots, 3 modules, 8 axes	**153** tests automatiques, tous verts (68 avant ce travail)	**0** octet changé dans les 94 fichiers générés de référence

Ce que le fichier permet maintenant

- **Changer une IP, une adresse, un logiciel de contrôleur** : on modifie le site dans le fichier, les projets liés reprennent les nouvelles valeurs à leur prochaine ouverture ; il reste à regénérer et téléverser.
- **Ajouter une carte, un module, un modèle de variateur** dont les puces ou le pilote existent déjà : une entrée d'une dizaine de lignes, plus le fichier `data/hardware` que lit le firmware. Plus aucune ligne de JavaScript.
- **Suivre le firmware** : version, limites (noms, nœuds, esclaves Modbus…), classes C++, environnements PlatformIO et bornes des variateurs sont lus dans le fichier ; quand le firmware relève une limite, on reporte la valeur et la validation applique le nouveau seuil.
- **Empêcher les erreurs d'intégration** : un profil incompatible avec le logiciel relevé sur le contrôleur est refusé. C'est exactement le piège du banc AUBO i10 (profil Modbus ARCS sur un contrôleur AUBOPE 4.5.13).
- **Contrôler la cohérence** : `plc infra verifier` compare le fichier à la vérité du firmware (`data/hardware/*.json`, `platformio.ini`, en-têtes C++) et nomme chaque écart avec sa ligne.

Ce qui ne change pas. Le firmware ne lit jamais ce fichier : il continue de lire `config.json` et `data/hardware/*.json`. Aucun fichier du firmware, de `data/` ni de `platformio.ini` n'a été modifié ; les incohérences trouvées de ce côté sont listées au chapitre 9 pour décision.

Garantie de non-régression

Le passage au référentiel s'est fait à résultat constant. Un banc d'équivalence a généré, avant et après, les fichiers de 11 projets (fixtures des tests, exemples du skill, `DemoPresse`) : les 94 fichiers sont identiques à l'octet, et le catalogue exposé par la ligne de commande aussi. Seule différence, voulue : deux avertissements nouveaux sur le projet d'essai « îlot robots » (« logiciel du contrôleur non vérifié » pour des robots AUBO et FAIRINO non liés à un site).

2 Démarche

Le travail a suivi quatre étapes, chacune confiée à plusieurs agents travaillant en parallèle, avec une vérification croisée avant de passer à la suivante.

Cartographie. Neuf lecteurs ont parcouru le catalogue, le générateur, le modèle et la validation, l'interface et le serveur, le firmware, les fichiers matériels, les tests et la documentation. Résultat : l'inventaire de chaque donnée d'infrastructure codée en dur (où, combien de copies, qui la lit) et 43 incohérences revérifiées dans le code.

Conception. Trois propositions indépendantes (point de vue du mainteneur, de la sûreté, de l'évolution) ont été jugées puis fusionnées : format JSONC commenté, sections rangées de la plus souvent modifiée à la plus rare, chargeur pur sans dépendance, façade `catalog.js` pour ne rien casser.

Implémentation par étapes, chacune validée par le banc d'équivalence : le fichier devient la source du catalogue ; générateur, validation et sites branchés dessus ; interface (choix du site, liaison des équipements) ; commandes `plc infra` ; vérificateur contre le firmware.

Relecture adversariale. Des relecteurs ont cherché à casser chaque partie (chargeur, sites, vérificateur, interface, documentation) ; 63 constats ont été confirmés par une contre-vérification, puis tous corrigés. Un test de robustesse a ensuite appliqué 36 567 mutations au fichier (valeurs nulles, types faux, clés manquantes, sur 2 151 chemins) : chacune produit un message d'erreur clair, aucune ne fait planter l'outil.

Principes retenus

Principe	Conséquence
Un seul fichier, lisible par un technicien	JSONC (JSON avec commentaires), clés en français, en-tête qui explique les notations, un exemple en commentaire pour chaque forme rare.
Le firmware reste maître de sa vérité	Le référentiel indexe `data/hardware` et `platformio.ini` sans les remplacer ; `plc infra verifier` contrôle qu'ils restent d'accord.
Une faute de saisie ne passe jamais en silence	Le chargeur refuse les clés en double, les types faux, les références inconnues, les nombres mal écrits ; chaque message cite `infrastructure.jsonc:ligne:colonne` et propose le nom le plus proche. Un fichier invalide arrête l'outil (code 3).
Ne jamais casser un projet existant	Les identifiants enregistrés dans les projets sont figés (`test/snapshots/identifiants.json`) ; un renommage passe par la section `alias`, sinon `npm test` échoue en expliquant pourquoi.
Pas de secret	Le chargeur refuse les clés de mot de passe et de WiFi : ces données restent dans le projet.

3 Le fichier `infrastructure.jsonc`

Les sections sont rangées de la plus souvent modifiée (les sites) à la plus rare (les écarts connus). Chaque section dit, en commentaire, ce qu'elle contient et contre quoi elle est vérifiée.

Section	Contenu	Contrôlé contre
`sites`	Appareils réellement installés : IP, `modbusTcp` (port, n° d'unité), adresse `rs485`, Ethernet et bus de l'automate, logiciel relevé (nom, version, date).	Cohérence interne : IP uniques, dans le sous-réseau, logiciel compatible.
`logiciels`	Logiciels de contrôleur connus (ARCS, AUBOPE, FAIRINO) et présence d'un serveur Modbus TCP.	—
`automates`	Une entrée par carte : environnement PlatformIO, Ethernet, horloge, voies, marque.	`data/hardware/<carte>.json`, `platformio.ini`.
`robots`, `modules`	Profils Modbus : voies, adresse et transport par défaut, logiciel requis.	`data/hardware/<profil>.json`.
`axes`	Modèles de variateur : pilote, famille, vitesses, rampe maximale, courant crête par modèle, format série imposé, plage d'adresses.	`data/hardware`, valeurs par défaut des pilotes C++ (`defaultsFor`).
`compilation`	Environnements PlatformIO : MCU, carte, flash, cœur Arduino, partitions.	`platformio.ini`, fichiers de partitions (casse comprise).
`firmware`	Version, 17 limites, types de section et classes C++, puces gérées, noms réservés.	En-têtes C++ (`#define`, `constexpr`, classes).
`pilotes`	Types d'axe que le **code** sait générer : classe, structure de nœuds, écran, signaux.	Structures `*Nodes` des pilotes C++.
`gabarits`	Morceaux réutilisables : voies d'automate, options, état de variateur, écran d'axe.	—
`alias`	Anciens noms encore acceptés (types, voies, options, sites, appareils).	—
`ecartsConnus`	Écarts acceptés avec le firmware, signalés en avertissement.	« Cliquet » : un écart corrigé doit être retiré.

Exemple : un modèle de variateur

Tout ce que le générateur et la validation doivent savoir du Lichuan A6 tient dans cette entrée. Les bornes (`rampeMaxMs`, `adresses`, `formatSerie`) étaient auparavant codées dans le générateur C++ ou absentes.

```
"LichuanA6": {
  "libelle": "Servo Lichuan A6 / LCDA6 (Modbus)",
  "fichierMateriel": "Lichuan_A6",
  "adresseParDefaut": 1,
  "pilote": "lichuan",
  "famille": "A6",
  "vitesseMax": 3000, "vitesseJogMax": 3000,
  "rampeMaxMs": 2500,                          // borne du firmware (LichuanOptions, PA_058)
  "formatSerie": { "impose": "SERIAL_8E1" },   // format fixe du variateur : tout le bus en 8E1
  "adresses": "1-32",                          // PA_000
  "noeuds": { "registres ecrits": [ { "id": 1, "nom": "entrees virtuelles", "champ":
"virtualInputs", … } ] }
}
```

Exemple : le site du banc AUBO

```
"banc-aubo": {
  "libelle": "Banc AUBO i10 (atelier)",
  "reseau": { "sousReseau": "192.168.10.0/24", "passerelle": "192.168.10.1" },
  "appareils": {
    "automate": {
      "type": "Waveshare_ESP32S3_POE_8DI8DO",
      "ethernet": { "ip": "192.168.10.20", "masque": "255.255.255.0", "passerelle":
"192.168.10.1" }
    },
    "robot": {
      "materiel": "AUBO i10, coffret CB-M",
      "logiciel": { "nom": "AUBOPE", "version": "4.5.13", "releveLe": "2026-10-11" },
      "ip": "192.168.10.50",
      "modbusTcp": null       // aucun serveur Modbus TCP sous AUBOPE : E/S câblées « linkage »
    }
  }
}
```

Notations

`"gabarit": "nom"`	Reprend un morceau de `gabarits` puis applique les clés locales (`null` retire une clé).
`"ids": "1-8"`	Plage d’identifiants (`"1-4, 9"` possible).
`"noms"` , `"broches"`	Noms des voies dans l’ordre des ids ; broches GPIO d’une carte.
`"noeuds": "par modèle"`	Dans un pilote : chaque modèle de la section `axes` donne ses propres nœuds.
`"seulementSi"`	Nœud présent seulement si le modèle a la propriété indiquée (ex. `boucleFermee`).

4 Comment PLC Studio s’en sert

Le fichier est lu une fois, au démarrage, par un chargeur unique (`shared/referentiel.js`). Le serveur, la ligne de commande et le navigateur travaillent ainsi sur exactement la même version.

Chargement

- **Serveur et ligne de commande** : `cli/demarrage.mjs` est importé en premier. Avec `--root <dépôt>` , l’outil lit le `infrastructure.jsonc` de ce dépôt ; `PLC_REFERENTIEL=<fichier>` fait lire un brouillon.
- **Navigateur** : la page reçoit le fichier depuis la mémoire du serveur (`/infrastructure.jsonc`), jamais depuis le disque : page et générateur ne peuvent pas diverger.
- **Façade** : `shared/catalog.js` expose toujours `CATALOG` (identique à l’octet à l’ancien), plus `REFERENTIEL` , `limite()` , `infoEquipement()` et les alias. Le reste du code n’a pas eu à changer de forme.

Ce qui vient désormais du fichier

Partie de PLC Studio	Données lues dans le référentiel
Catalogue (interface, CLI, skill)	Équipements, voies, options et leurs choix, libellés et aides.
Validation (`validate.js`)	Limites du firmware (taille des noms, nombre de nœuds, esclaves Modbus, IP TCP…), classes C++ et noms de projet réservés, plage d’adresses et format série imposé par modèle, compatibilité logicielle des sites.
Générateur C++ (`cpp.js`)	Bornes par modèle : rampe maximale, rampe par défaut, courant crête (DM556RS 5,6 A, DM882RS 8,2 A, CL57RS 7,0 A, CL86RS 8,0 A au lieu d’un 8,2 A unique).
Mise en page (`layout.js`)	Classe C++ de chaque type de section, longueur maximale des noms, port Modbus TCP, nœuds par section.
Écrans d’axe (`hmi.js` , `spec.js`)	Écran automatique d’un axe et alias de ses valeurs (`gabarits.ecranAxe`).
`plc check`	Commande de compilation propre à la carte du projet (`pio run -e <env> -t upload` , puis `-t uploadfs`), et le référentiel utilisé (chemin, révision, empreinte).

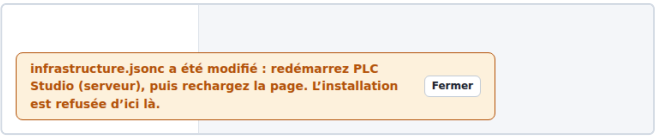
Erreur de saisie

Une faute dans le fichier bloque le démarrage avec un message précis, sans laisser l’outil travailler sur des données fausses :

```
$ node tools/plc-studio/cli/plc.mjs infra verifier
Référentiel d'infrastructure invalide : 1 erreur(s)
  infrastructure.jsonc:144:19  valeur inattendue « vrai » (écrire true ou false)
(code de sortie 3)
```

Fichier modifié pendant que PLC Studio tourne

Un bandeau prévient l'utilisateur (vérifié au retour sur l'onglet et toutes les 30 s), et le serveur **refuse l'installation** tant qu'il n'a pas été redémarré : on n'installe jamais un projet généré avec un référentiel périmé.



5 Sites, appareils et logiciels

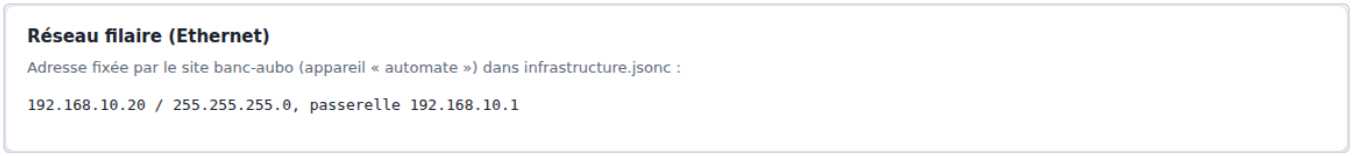
Un projet peut être rattaché à un site, et chacun de ses équipements à un appareil de ce site. Les adresses viennent alors du référentiel, et non plus du projet.

Ce que le site impose

- Pour un équipement Modbus lié : IP, port et n° d'unité (Modbus TCP) ou adresse RS485.
- Pour l'automate lié : son Ethernet (IP fixe, masque, passerelle) et, si le site le précise, la vitesse et le format du bus RS485.
- Ces champs apparaissent grisés avec leur provenance. Quand le site cesse d'imposer une valeur, le champ redevient libre ; délier un équipement ou retirer le site garde les dernières valeurs. Un projet sans site n'est jamais modifié.



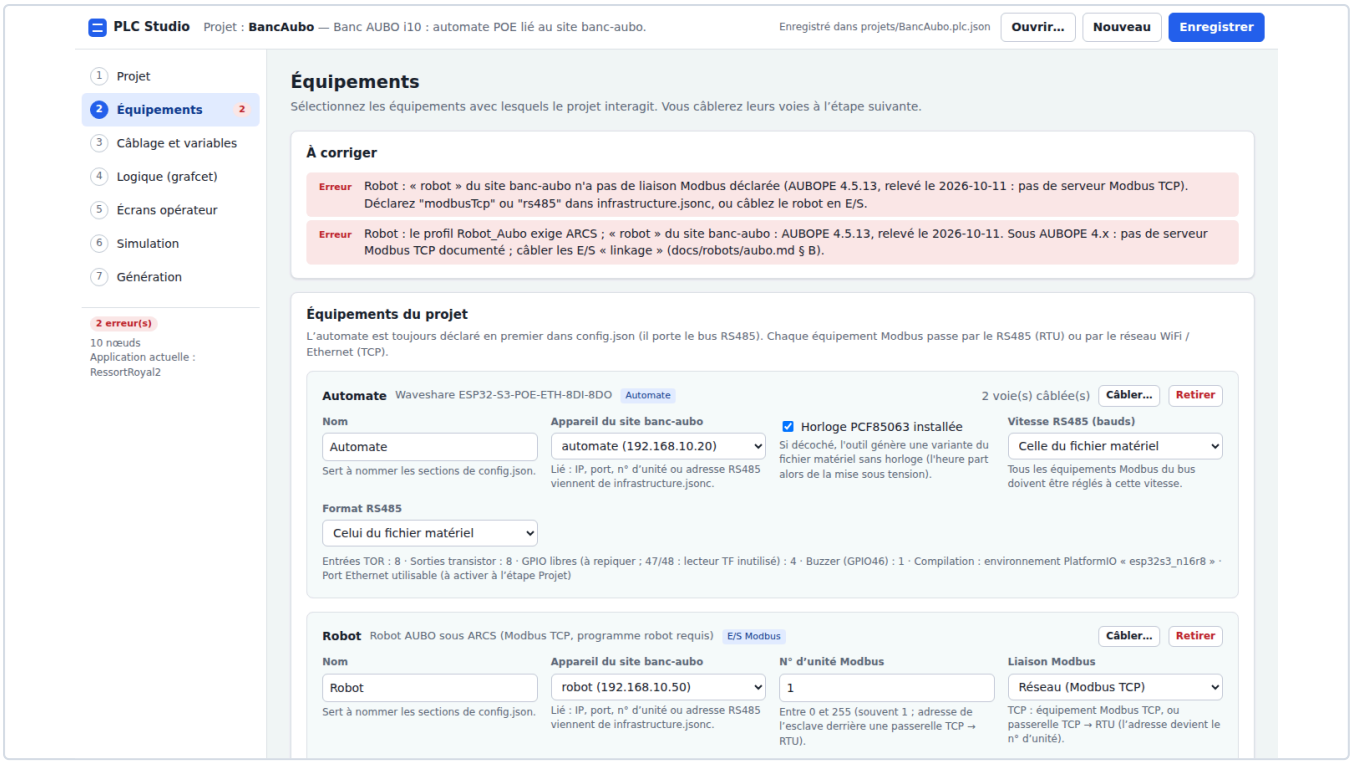
Étape Projet : choix du site et appareils du banc AUBO, avec le logiciel relevé sur le robot.



L'Ethernet de l'automate vient du site.

Compatibilité logicielle : le piège du banc AUBO évité

Le profil `Robot_Aubo` exige un contrôleur ARCS (`"logicielRequis": { "noms": ["ARCS"] }`). Le robot du banc a été relevé sous AUBOPE 4.5.13, sans serveur Modbus TCP. Un projet qui relie ce profil à ce robot est désormais refusé, avec l'explication et la marche à suivre — exactement l'erreur qui avait fait échouer la communication lors du diagnostic du 11 octobre.



Étape Équipements : le robot AUBO lié au site est refusé (AUBOPE 4.5.13 sans serveur Modbus TCP) ; l'automate est lié à l'appareil « automate ».

Même contrôle en ligne de commande, avec la commande de compilation propre à la carte :

```
$ node tools/plc-studio/cli/plc.mjs build banc.spec.json
Vérification : 2 erreur(s), 0 avertissement(s) — 10 nœuds
  ERREUR [Équipements] Robot : « robot » du site banc-aubo n'a pas de liaison Modbus déclarée
         (AUBOPE 4.5.13, relevé le 2026-10-11 : pas de serveur Modbus TCP). …
  ERREUR [Équipements] Robot : le profil Robot_Aubo exige ARCS ; « robot » du site banc-aubo :
         AUBOPE 4.5.13, relevé le 2026-10-11. Sous AUBOPE 4.x : … câbler les E/S « linkage ».
Installable : non (corriger les erreurs).
Compilation (Waveshare ESP32-S3-POE-ETH-8DI-8DO) : pio run -e esp32s3_n16r8 -t upload, puis -t
uploadfs pour data/
Référentiel : infrastructure.jsonc — 2026-10-15 : création ; banc AUBO … (9c83bd62)
```

Un robot dont le profil exige un logiciel, mais qui n'est lié à aucun site, reçoit seulement un avertissement « logiciel du contrôleur non vérifié ».

6 Les outils `plc infra`

Commande	Rôle
`infra verifier [--strict]`	Contrôle le référentiel contre `data/hardware`, `platformio.ini` et les en-têtes C++. Code 1 s'il reste une erreur (avec `--strict` : aussi un avertissement).
`infra liste`	Automates (compilation, Ethernet, horloge, GPIO libres), robots, modules, axes, environnements, sites et appareils.
`infra montrer <type|site[.app]>`	Entrée développée telle que PLC Studio la voit (gabarits appliqués), avec sa ligne dans le fichier.
`infra impact <site>[.<app>] [--ip <ancienne>]`	Projets liés au site ou à l'appareil, et projets non liés qui contiennent une de ses IP : à lancer **avant** de changer une IP.
`infra figer`	Met à jour la copie figée des identifiants après un ajout relu ou un renommage voulu.

Ce que vérifie `infra verifier`

- **Fichiers matériels** : existence, nom, chaque id de chaque voie, taille des bancs, broches GPIO et buzzer, bloc Ethernet présent si et seulement si annoncé, bus RS485, horloge.
- **`platformio.ini`** : environnements, carte, flash, cœur Arduino (Ethernet ⇒ Arduino 3), fichiers de partitions (casse comprise), environnement par défaut.
- **Firmware** : version, limites (`#define`, `constexpr`), types de section et classes C++, puces gérées, structures de nœuds des pilotes (nom et type de chaque champ), bornes des variateurs lues dans `defaultsFor`.
- **Identifiants** : nouveaux équipements pas encore figés.

Un symbole introuvable est une erreur nommée, jamais un contrôle sauté en silence. Résultat actuel sur le dépôt :

```
$ node tools/plc-studio/cli/plc.mjs infra verifier
AVERTISSEMENTS (4)
  infrastructure.jsonc:123  H7  [écart connu E07] Kincony_KC868_A8S.json : RS485 à 115000 bauds
…
  infrastructure.jsonc:544  P2  [écart connu E14] platformio.ini cite partitions_16MB.csv, …
  infrastructure.jsonc:418  F6  [écart connu E02] LichuanA5 vitesseMax = 6000 ici, 3000 dans le
firmware
  infrastructure.jsonc:418  F6  [écart connu E02] LichuanA5 vitesseJogMax = 6000 ici, 3000 …
INFORMATIONS (6) …
… : 0 erreur(s), 4 avertissement(s), 6 information(s)
```

Écarts connus

Quand le référentiel et le firmware divergent volontairement (en attendant une décision), l'écart est inscrit dans `ecartsConnus` avec le contrôle qu'il couvre et un **motif** (extrait du message attendu) : seul ce constat devient un avertissement, tout autre problème au même endroit reste une erreur. Quand l'écart est corrigé dans le firmware, le vérificateur demande de le retirer.

7 Procédures de changement

Le détail pas à pas est dans `docs/referentiel-infrastructure.md`. Légende : **R** = seul `infrastructure.jsonc` change ; **R+F** = un fichier lu par le firmware ou PlatformIO change aussi (inévitable, ils ne lisent pas le référentiel) ; **C** = du code doit changer.

Changement	Où	Étapes principales
IP ou n° d'unité d'un robot, d'une passerelle	**R**	`infra impact` d'abord ; modifier l'appareil du site ; `infra verifier` ; regénérer, installer et téléverser `data/` pour les projets liés.
Logiciel du robot relevé	**R**	Renseigner `logiciel` (nom, version, date) et `"modbusTcp": null` sans serveur Modbus : PLC Studio refuse alors un profil incompatible.
Bus RS485 d'un site	**R**	`appareils.automate.bus = { "vitesse": 19200, "format": "SERIAL_8N1" }` ; un variateur à format imposé (A6 : 8E1) est refusé sur un autre format.
Remplacer la carte d'un banc par une carte connue	**R + projet**	Changer le `type` de l'appareil « automate » ; dans les projets liés, changer l'automate et le relier à l'appareil ; recâbler les voies signalées.
Ajouter une carte (puces déjà gérées)	**R+F**	Fichier `data/hardware` ; entrée `automates` d'une dizaine de lignes ; `infra verifier`, `npm test`, `infra figer`. Une nouvelle puce d'extension, d'horloge ou d'Ethernet demande du code (C).
Révision de carte (GPIO déplacé)	**R+F**	Modifier le fichier matériel **et** `"broches"` : le vérificateur contrôle qu'ils concordent.
Nouveau modèle d'une série de variateur connue	**R (+F)**	Entrée `axes` : pilote, famille, bornes, adresses, format imposé ; `infra verifier` ; `infra figer`.
Le firmware relève une limite	**C, puis R**	Modifier le C++ ; le vérificateur signale l'écart ; reporter la valeur (et `firmware.version`).
Nouvel environnement PlatformIO	**F, puis R**	`[env:…]` et partitions ; `compilation.envs` ; `envPlatformIO` des cartes concernées.
Renommer un type, une voie, un site, un appareil	**R**	Renommer et garder l'ancien nom dans `alias` : projets et specs restent valides. Les clés lues par le code (options `ip`, `model`… ; voies `q`, `i`…) et les noms qui entrent dans le hash des nœuds ne se renomment pas ainsi.
Nouveau type d'axe	**C + F + R**	Classe C++, génération et simulation dans PLC Studio, puis `pilotes` et `axes`. Sans le code, le chargeur refuse le pilote et le dit.

Après chaque modification : `node tools/plc-studio/cli/plc.mjs infra verifier`, puis `npm test` dans `tools/plc-studio`, puis redémarrer PLC Studio s'il tourne.

8 Vérifications effectuées

Vérification	Résultat
Tests automatiques (`npm test`)	**153 / 153** réussis (68 avant), dont 85 nouveaux : chargement et messages d'erreur des fautes courantes, identifiants figés, alias, sites (propagation, compatibilité, retrait), vérificateur (mutations du firmware simulées), génération de chaque équipement avec les vrais fichiers matériels.
Équivalence avant / après	94 fichiers générés pour 11 projets : identiques à l'octet. Catalogue et sortie de `plc catalog` identiques. Seul ajout : 2 avertissements « logiciel non vérifié » sur un projet d'essai.
Syntaxe du C++ généré (`npm run check:cpp`)	OK pour les 4 projets de contrôle (axes multiples SureServo et pas-à-pas, Lichuan A6/A5, cellule robot Modbus TCP + Ethernet, îlot robots + StepperOnline).
Robustesse du chargeur	36 567 mutations du fichier sur 2 151 chemins : 0 plantage, chaque cas produit un message situé.
Vérificateur sur le dépôt	0 erreur, 4 avertissements (écarts connus E02 ×2, E07, E14), 6 informations.
Interface (navigateur automatisé)	Choix du site, liaison des appareils, champs imposés, Ethernet du site, erreurs de compatibilité, bandeau et refus d'installation après modification du fichier : vérifiés, sans erreur JavaScript.
Relecture adversariale	63 constats confirmés, tous corrigés et couverts par un test quand c'était possible.

Non fait. Aucune compilation ESP32 ni essai sur un automate : comme convenu, la compilation PlatformIO se fait sur le poste. Rien ne change pour le firmware dans cette branche.

9 Points à décider (firmware et fichiers matériels)

La cartographie et le vérificateur ont révélé des incohérences du côté du firmware, de `data/` et de `platformio.ini`. Ces fichiers n'ont pas été modifiés : chaque point attend votre décision.

Inscrits comme écarts connus (signalés à chaque vérification)

Écart	Constat	Correction proposée
E02	Lichuan A5 annoncé à 6000 tr/min (vitesse et jog) ; le firmware borne à 3000 (`maxSpeedRpm` jamais relevé pour l'A5), alors que son propre commentaire dit « A5 6000 ».	Relever `maxSpeedRpm` de l'A5 dans `defaultsFor`, ou ramener le référentiel à 3000.
E07	`data/hardware/Kincony_KC868_A8S.json` règle le RS485 à **115000** bauds (les 7 autres cartes : 115200). La documentation des variateurs demande 115200 : probable faute de frappe.	Corriger à 115200.
E14	`platformio.ini` (env `esp32dev`) cite `partitions_16MB.csv`, le fichier s'appelle `partitions_16Mb.csv` : la compilation échoue sous Linux. `partitions_16MB_no_ota.csv` (env de test) est introuvable.	Renommer le fichier ou corriger la ligne.
E15	Environnement de l'A8S : `esp32dev` dans le dépôt, `quatre_mb_huge` en déploiement réel (audit MA-31).	Choisir, puis mettre à jour `envPlatformIO`.

Autres constats

Sujet	Constat
Inversion par nœud	L'inversion d'une entrée `ModbusReadCoil` par nœud est ignorée par le firmware (lue seulement à la racine du fichier matériel).
`MODBUSRTU_TIMEOUT`	`-DMODBUSRTU_TIMEOUT=100` est sans effet : la bibliothèque modbus-esp8266 redéfinit 1000 sans condition.
`NAME_LENGHT`	Défini deux fois (80 dans `Node.h` et `borneUniverselle.h`) ; 78 octets sûrs en pratique (PLC Studio l'applique).
Buzzer de l'A8S	Le firmware retombe sur le nom du fichier pour trouver le buzzer ; la broche est aussi une broche de démarrage.
Horloges	A8S, A16v3, A8v3 : `RTC` à `true` ou `"auto"` sans nommer la puce, donc invérifiable.
Extensions A16 / A16v3	Ordre des adresses d'extension inversé entre les deux cartes : à vérifier sur les schémas.
StepperOnline W8	Registre `quickStopTime` déclaré dans les fichiers et la structure C++, jamais câblé.
`config.json`	Le test du firmware sur `type` est inversé : seul le type numérique fonctionne.
Commentaire `MyModbus.h`	« throttle 30s » alors que l'intervalle réel est 5 s.
Bibliothèques	Versions d'ArduinoJson, AsyncTCP et ESPAsyncWebServer différentes selon l'environnement ; FastAccelStepper ^0.33 alors que la documentation annonce ^2.0.
Mise à jour à distance (OTA)	Toutes les tables de partitions sont « no_ota » : une mise à jour à distance n'est pas possible en l'état.
Stub de test `Arduino.h`	Le format série 8O1 proposé n'est pas défini dans le stub (sans effet aujourd'hui).

Sécurité. `data/config.json` et `src/WifiManagement/wifimanagment.h` contiennent de vrais identifiants WiFi. Ils n'ont jamais été recopiés dans le référentiel (le chargeur refuse ces clés) ; il est conseillé de les retirer de l'historique Git et de changer les mots de passe concernés.

10 Fichiers, branche et commits

Branche `claude/referentiel-infrastructure`, partie de `claude/cartes-et-axes` (`5816df0`), poussée sur `origin` avec ce rapport (`Rapport_referentiel_infrastructure.pdf`, à la racine). Aucune demande de fusion n'a été ouverte.

Commits

`5d82e28`	`infrastructure.jsonc` devient la source du catalogue.
`f1deab0`	Générateur, validation et sites branchés sur le référentiel.
`81ddc07`	Interface : choix du site et liaison des équipements aux appareils.
`2a019bd`	Commandes `plc infra`, documentation des procédures.
`dcccdfb`	Vérificateur `plc infra verifier` contre la vérité du firmware.
`97a59b4`	Corrections issues de la relecture (chargeur, sites, vérificateur, interface).
`f3c8242`	Documentation (impact avant changement d'IP, figer, bus, `--root`, écarts).

Fichiers principaux

Fichier (code : sous `tools/plc-studio/`)	Rôle
`infrastructure.jsonc`	**Le référentiel** : le seul fichier à modifier quand l'infrastructure change.
`docs/referentiel-infrastructure.md`	Mode d'emploi et procédures pas à pas.
`shared/referentiel.js`, `shared/jsonc-pos.js`	Chargeur (gabarits, contrôles, messages situés) et lecteur JSONC strict qui garde la ligne de chaque clé.
`shared/catalog.js`, `shared/site.js`	Façade (`CATALOG`, `REFERENTIEL`, `limite()`, alias) ; sites (valeurs imposées, compatibilité logicielle).
`shared/referentiel-verif.js`, `shared/identifiants.js`	Vérificateur contre `data/hardware`, `platformio.ini` et le C++ ; identifiants figés et contrôle des renommages.
`cli/infra.mjs`, `cli/demarrage.mjs`	Commandes `plc infra` ; chargement commun et choix du fichier lu.
`test/referentiel*.test.js`, `test/sites.test.js`	85 nouveaux tests ; jeu d'essai figé `test/donnees/infrastructure-essai.jsonc`.
Interface : `public/js/views/project.js`, `equipment.js`, `app.js`	Carte « Site », champ « Appareil du site », champs imposés, bandeau.
Documentation mise à jour	`README.md`, `tools/plc-studio/README.md`, `docs/cartes.md`, `docs/modbus-tcp.md`, skill `plc-projet` (`SKILL.md`, `references/spec.md`).

Prochaines étapes suggérées

1. Trancher les écarts E02, E07, E14, E15 (chapitre 9), puis les retirer du référentiel.
2. Fusionner la branche après relecture ; ensuite, toute évolution de l'infrastructure passe par `infrastructure.jsonc` et `plc infra verifier`.